Department of Computer Science

Course Code: CIC201

Course Title: Artificial Intelligence

Fall 2025

Lab 10

Introduction to Unsupervised Learning

Objective:

This lab provides a comprehensive Introduction about different Unsupervised learning algorithm

Name of Student	
Student ID	
Date of Lab Conducted	
Marks Obtained	
Remarks	
Signature	

1. Objective

This lab provides a comprehensive Introduction about different Unsupervised learning algorithm

.

2. Instructions

You have to perform the following tasks yourselves. Raise your hand if you face any difficulty in understanding and solving these tasks. Plagiarism is an abhorrent practice and you should not engage in it.

3. How to Submit

Submit lab work using TEAMS

Unsupervised machine learning

Introduction:

Unsupervised machine learning is a fascinating field that involves training models on data without explicit supervision or labeled outcomes. Unlike supervised learning, where the algorithm learns from a labeled dataset, unsupervised learning explores the hidden patterns and structures within unlabeled data. One of the most widely used unsupervised learning algorithms is K-means clustering**.**

Unsupervised Learning:

In unsupervised learning, the algorithm explores the inherent structure of the data without the guidance of predefined labels. This type of learning is particularly useful when we want to discover patterns, relationships, or groups within the data. Common techniques in unsupervised learning include clustering, dimensionality reduction, and generative modeling.

K-means Clustering K-

means Clustering:

K-means clustering is a popular unsupervised learning algorithm used for partitioning a dataset into a set of distinct, non-overlapping groups or clusters. The "K" in K-means represents the number of clusters the algorithm aims to identify in the data. The algorithm iteratively assigns data points to clusters based on their similarity and updates cluster centers until convergence**.**

Implementation:

Generate Data

```
import numpy as np
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

# Set the number of clusters for the algorithm
num_clusters = 2  # Change this number to set the desired number of clusters

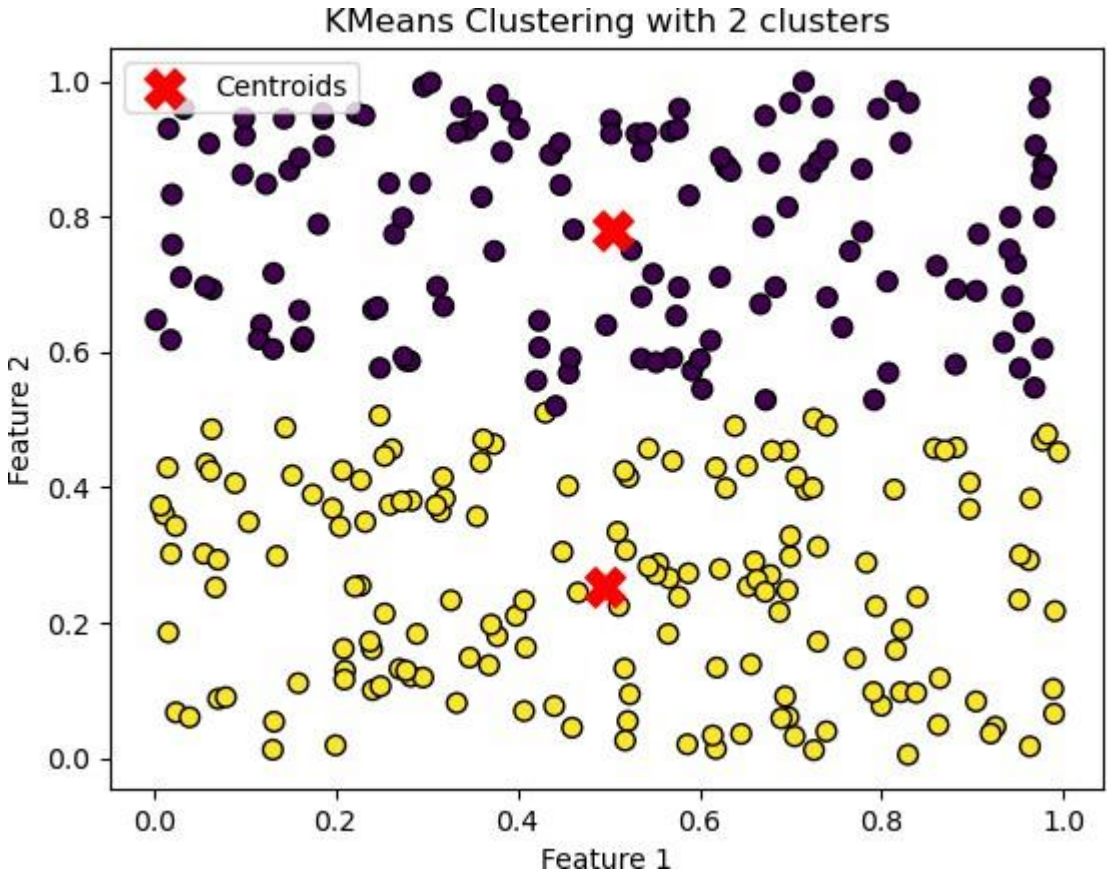
# Generate random data points
np.random.seed(0)
#Generates a 2D array (matrix) with 300 rows and 2 columns,
X = np.random.rand(300, 2)
```

Apply K-means Clustering

```
# Perform KMeans clustering
kmeans = KMeans(n_clusters=num_clusters, random_state=0)
labels = kmeans.fit_predict(X)
```

Visualize the Results

```
# Plot the clustered data points
plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='viridis', edgecolor='k', s=50)
plt.scatter(kmeans.cluster_centers_[:, 0], kmeans.cluster_centers_[:, 1], c='red', marker='X', s=200, label='Centroids')
plt.title(f'KMeans Clustering with {num_clusters} clusters')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.legend()
plt.show()
```



make_blobs In Python:

The make_blobs function in scikit-learn is a utility for generating synthetic datasets with blobs of points belonging to different classes. This function is often used for testing and illustrating clustering algorithms.

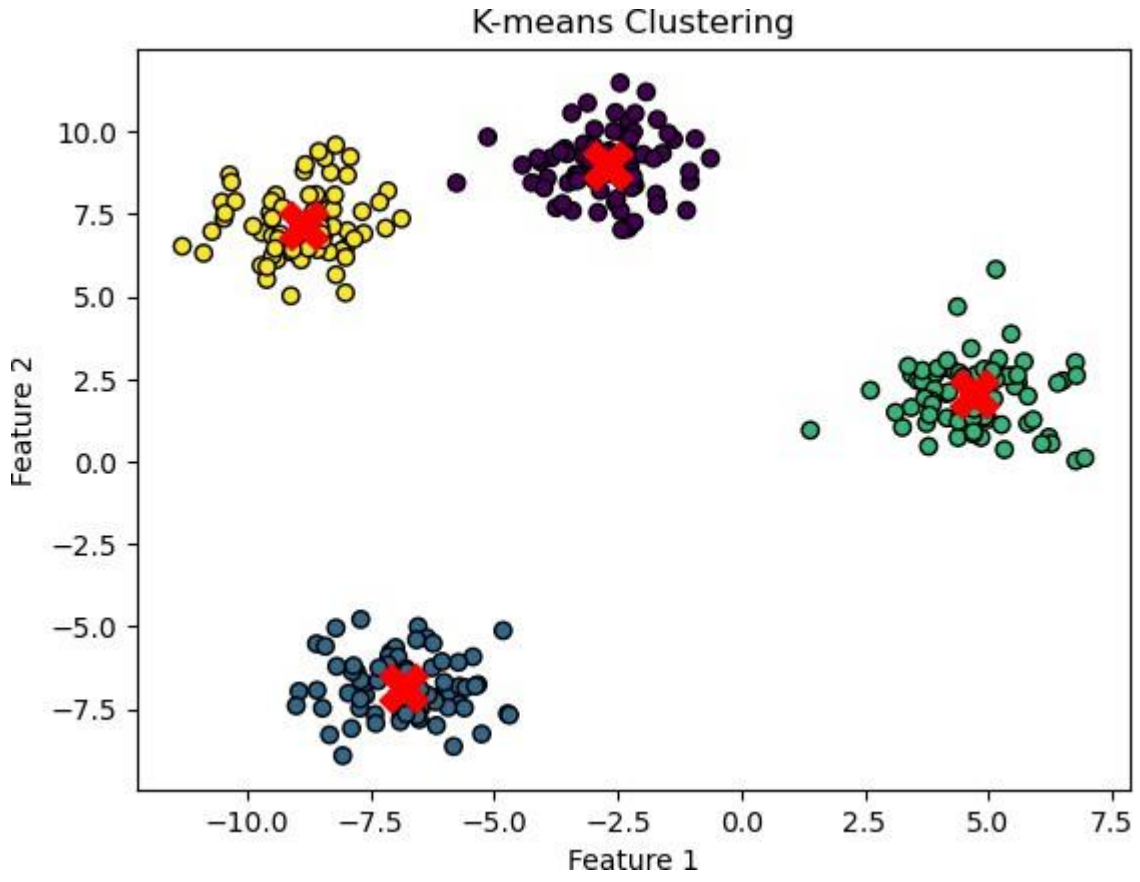
This function is commonly used in examples to show the behavior of clustering algorithms, as it creates clearly separated clusters that make it easy to visualize the algorithm's performance.

```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs

# Generate random data
np.random.seed(42)
X, _ = make_blobs(n_samples=300, centers=4, random_state=42)

# Apply K-means clustering
kmeans = KMeans(n_clusters=4)
kmeans.fit(X)
y_kmeans = kmeans.predict(X)

# Plot the data and cluster centers
plt.scatter(X[:, 0], X[:, 1], c=y_kmeans, cmap='viridis', edgecolor='k')
plt.scatter(kmeans.cluster_centers_[:, 0], kmeans.cluster_centers_[:, 1], s=300, c='red', marker='X')
plt.title('K-means Clustering')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.show()
```



PCA:

Principal Component Analysis (PCA) is a dimensionality reduction technique commonly used in machine learning and statistics. Its primary goal is to transform a dataset into a new coordinate system, where the features (or dimensions) are linearly uncorrelated, and the most important information is captured in the first few dimensions, known as principal components.

Center the Data:

Subtract the mean of each feature from the dataset, so that the data is centered around the origin.

Formula:

$$X_{\text{centered}} = X - \bar{X}$$

where X_{centered} is the centered data matrix, X is the original data matrix, and $\bar{X}$ is the mean of each feature.

Calculate Covariance Matrix:

Compute the covariance matrix, which represents the relationships between different features in the dataset

Formula:

$$\text{Cov}(X) = \frac{1}{n-1}(X_{\text{centered}}^T \cdot X_{\text{centered}})$$

where n is the number of samples.

Eigenvalue Decomposition:

Find the eigenvalues and eigenvectors of the covariance matrix. The eigenvectors represent the directions (principal components), and the eigenvalues represent the magnitude of variance in those directions.

Formula:

$$\text{Cov}(X) \cdot \mathbf{v} = \lambda \cdot \mathbf{v}$$

where $\mathbf{v}$ is the eigenvector and λ is the corresponding eigenvalue.

Sort and Select Principal Components:

Sort the eigenvectors by their corresponding eigenvalues in descending order. The eigenvector with the highest eigenvalue is the first principal component, the second highest is the second principal component, and so on. You can choose to keep only a subset of the principal components based on the amount of variance you want to retain.

Formula:

$$\mathbf{W} = \begin{bmatrix} \mathbf{v}_1 & \mathbf{v}_2 & \dots & \mathbf{v}_k \end{bmatrix}$$

where $\mathbf{W}$ is the transformation matrix containing the selected eigenvectors

Transform the Data:

Project the original data onto the selected principal components to obtain a lower-dimensional representation of the dataset.

Formula:

$$X_{\text{pca}} = X_{\text{centered}} \cdot \mathbf{W}$$

where X_{pca} is the reduced-dimensional representation of the data.

```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.datasets import load_iris
from sklearn.decomposition import PCA

# Load the Iris dataset
iris = load_iris()
X_iris = iris.data
y_iris = iris.target

# Print the original data
print("Original Data (first 5 samples):\n", X_iris[:5])

# Set the number of clusters for the algorithm
num_clusters = 3  # Change this number to set the desired number of clusters

# Perform KMeans clustering
kmeans = KMeans(n_clusters=num_clusters, random_state=1)
labels = kmeans.fit_predict(X_iris)

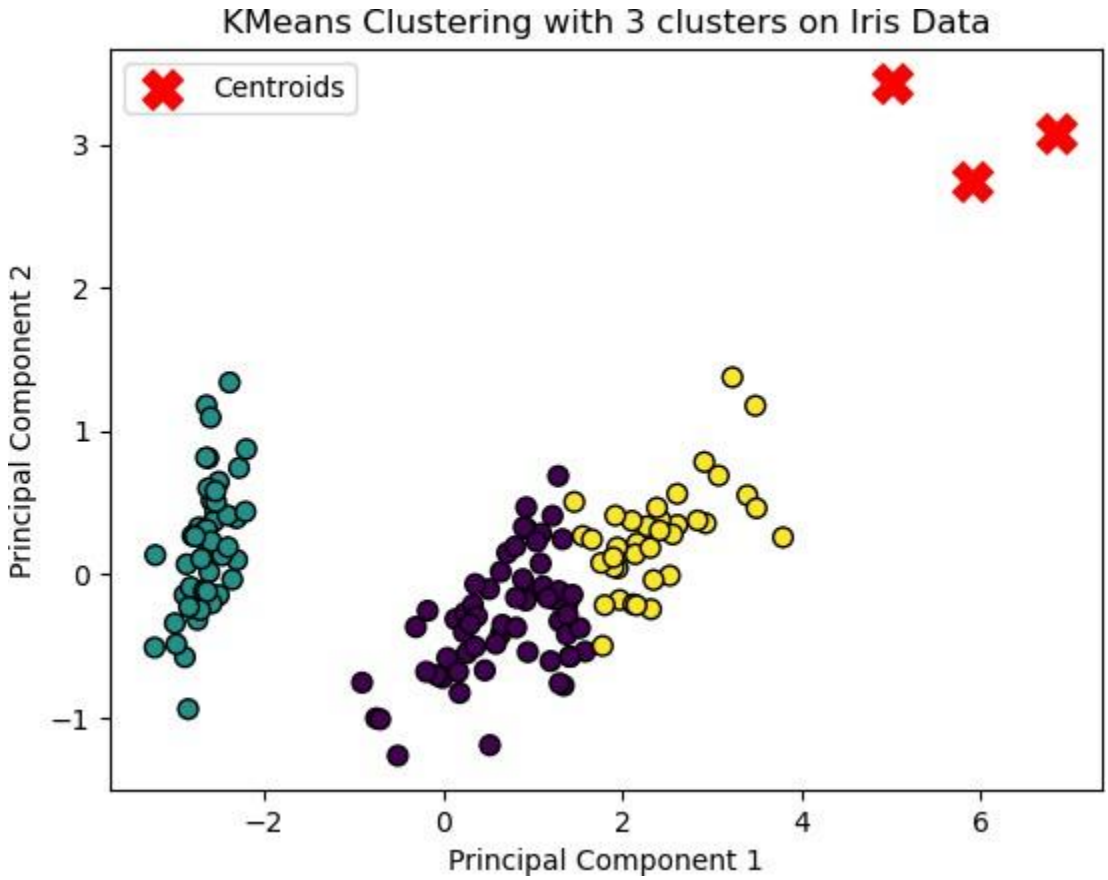
# Reduce the data to 2D for visualization (using PCA)
pca = PCA(n_components=2)
X_2d = pca.fit_transform(X_iris)

# Print the data after PCA
print("\nData after PCA (first 5 samples):\n", X_2d[:5])

# Plot the clustered data points
plt.scatter(X_2d[:, 0], X_2d[:, 1], c=labels, cmap='viridis', edgecolor='k', s=50)
plt.scatter(kmeans.cluster_centers_[:, 0], kmeans.cluster_centers_[:, 1], c='red', marker='X', s=200, label='Centroids')
plt.title(f'KMeans Clustering with {num_clusters} clusters on Iris Data')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.legend()
plt.show()
```

```
Original Data (first 5 samples):
 [[5.1 3.5 1.4 0.2]
 [4.9 3.  1.4 0.2]
 [4.7 3.2 1.3 0.2]
 [4.6 3.1 1.5 0.2]
 [5.  3.6 1.4 0.2]]

Data after PCA (first 5 samples):
 [[-2.68412563  0.31939725]
 [-2.71414169 -0.17700123]
 [-2.88899057 -0.14494943]
 [-2.74534286 -0.31829898]
 [-2.72871654  0.32675451]]
```



StandardScaler

The StandardScaler is a preprocessing technique used in machine learning to standardize or normalize the features of a dataset. In the context of clustering, and more specifically, the KMeans algorithm, using StandardScaler can be beneficial for various reasons.

Standardization of Features:

KMeans clustering is sensitive to the scale of features. If features have different scales, the algorithm might give more weight to features with larger scales, leading to biased cluster assignments. StandardScaler standardizes features by removing the mean and scaling to unit variance. This ensures that all features contribute equally to the distance computations performed by KMeans.

Improving Convergence:

Standardizing features can help KMeans converge faster. It reduces the differences in the ranges of features, allowing the algorithm to reach convergence more efficiently.

Enhanced Interpretability:

Standardization makes the interpretation of cluster assignments more meaningful. Without standardization, the clusters might be driven by the features with larger scales, which may not necessarily represent the most important aspects of the data.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

# Load the Iris dataset
iris = load_iris()
X_iris = iris.data
y_iris = iris.target

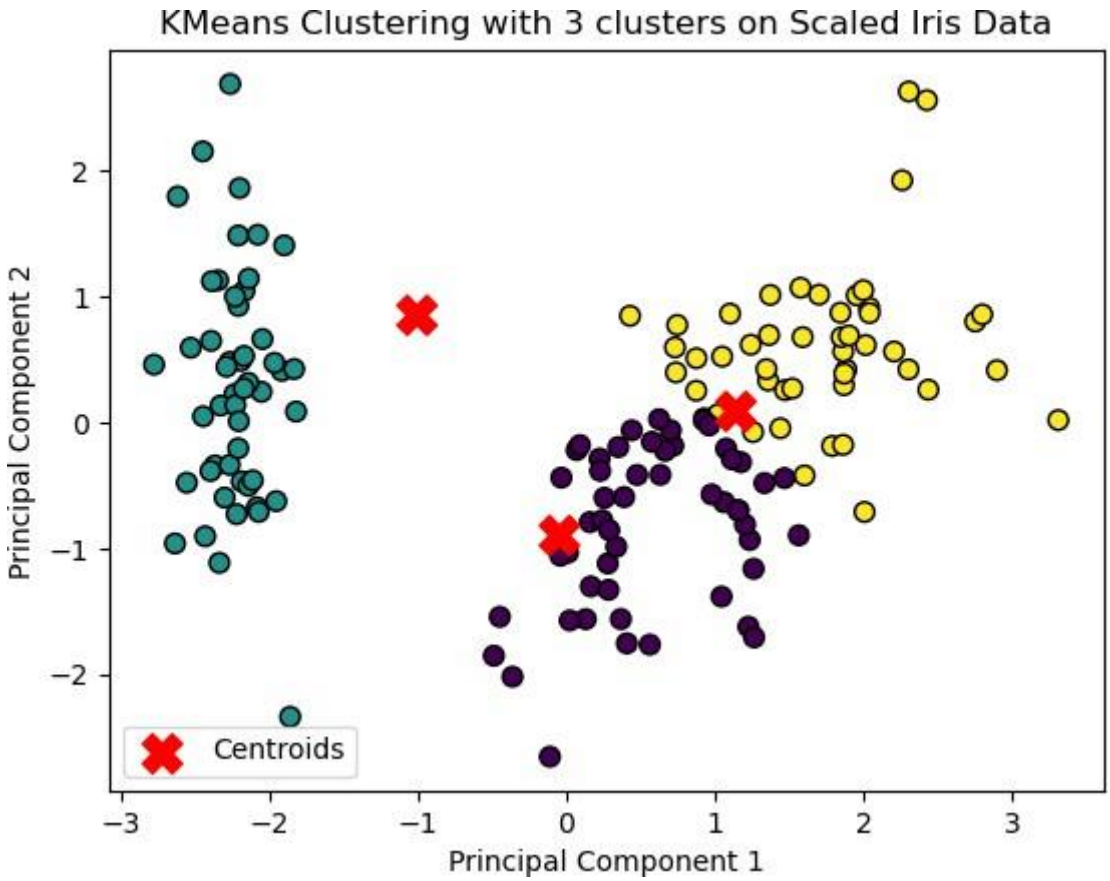
# Scale the data using StandardScaler
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_iris)

# Set the number of clusters for the algorithm
num_clusters = 3

# Perform KMeans clustering on the scaled data
kmeans = KMeans(n_clusters=num_clusters, random_state=0)
labels = kmeans.fit_predict(X_scaled)

# Reduce the data to 2D for visualization (using PCA)
pca = PCA(n_components=2)
X_2d = pca.fit_transform(X_scaled)

# Plot the clustered data points
plt.scatter(X_2d[:, 0], X_2d[:, 1], c=labels, cmap='viridis', edgecolor='k', s=50)
plt.scatter(kmeans.cluster_centers_[:, 0], kmeans.cluster_centers_[:, 1], c='red', marker='X', s=200, label='Centroids')
plt.title(f'KMeans Clustering with {num_clusters} clusters on Scaled Iris Data')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.legend()
plt.show()
```


KMeans Clustering with 3 clusters on Scaled Iris Data
Principal Component 2
Principal Component 1
Centroids
2
1
0
−1
−2
−3
−2
−1
0
1
2
3

Lab Task

Task 1: Exploratory Data Analysis (EDA) and K-means Clustering

Load the Iris dataset from scikit-learn (load_iris). Perform exploratory data analysis to understand the structure of the dataset.

Use K-means clustering to identify clusters in the Iris dataset. Experiment with different values of K and visualize the results using scatter plots.

Apply the Elbow Method to determine the optimal number of clusters for the Iris dataset.

Task 2: Dimensionality Reduction and Visualization

Load the Wine dataset from scikit-learn (load_wine). Apply Principal Component Analysis (PCA) to reduce the dimensionality of the dataset to two components.

Visualize the reduced Wine dataset in a 2D scatter plot, coloring the points based on their class labels.

Experiment with different clustering algorithms (e.g., K-means, hierarchical clustering) on the reduced Wine dataset and compare the results.

Task 3: Hierarchical Clustering

Load the Breast Cancer dataset from scikit-learn (load_breast_cancer). Apply hierarchical clustering to identify clusters. Visualize the resulting dendrogram and discuss the potential number of clusters.

Task 4: Density-Based Clustering

Load the Digits dataset from scikit-learn (load_digits). Apply DBSCAN (Density-Based Spatial Clustering of Applications with Noise) on the dataset. Discuss the strengths and weaknesses of DBSCAN compared to K-means.